Operator-Controlled MCP: A Security Layer for AI Agent Operations on Networks

Marc Lawson

Scuola Superiore Sant'Anna, Pisa, Italy

marc.lawson@santannapisa.it, monmaconair@gmail.com

LLM agents are starting to get write access to real infrastructure. On a network controller one wrong call can empty a flow table, make a switch drop all its traffic, or leave a circuit half built, and none of it can be taken back. As agents begin to operate networks, the operator needs a layer between the agent and the network where what the agent does is checked, shown and approved, and a stop that does not depend on the agent choosing to stop.

We present an operator-controlled MCP [1] server for the Ryu OpenFlow controller [2], developed in the scope of this work, as that layer. The agent can read the network freely, but every change it asks for is put under scrutiny: it is rated for risk, put to the user, approved by an operator, checked on the switch and written to a log. A person decides, and only then is anything sent. Fig. 1 shows the path. The sections below list what the server offers, how we tested it and what we found. Code and tests: `github.com/maconair0/ryu-mcp-server`.

What the server offers

An agent can freely read the switches, links, hosts, flows and port counters, and the shortest path between two switches. Writing is where the server holds back.

It cannot directly write: The write tools (add or delete a flow, map or unmap a VLAN) check the request against OpenFlow's rules [3] and put it in a queue. Nothing reaches Ryu until an operator approves it, on the command line or through a secret token that a host holds and the model never sees. No tool has a flag such as `force` or `approved` that skips the queue, because a flag the model can set is a flag the model will set. Each request is rated medium, high or critical, with the reasons, from what it does to the switch. A flow that drops every packet is critical, and so is deleting the flow that sends new traffic to the controller. Before a high or critical change is even queued, the server asks the person behind the client, using MCP's elicitation [4]. The model cannot answer for the user, because the answer is not an argument of any tool, and it cannot soften the question, because the server writes it. A no, a dismissed question or five minutes of silence queues nothing, and a critical change also needs a short code typed in. A critical change stays in the queue even when the server is set to apply changes directly.

Ryu answers 200 to flows that Open vSwitch then refuses, so the server reads the flow table back (Fig. 2). A flow the switch does not hold is reported failed, and whatever was already sent is taken back. The flows a change replaces are read before it is sent, so an undo restores them exactly. The undo is a new request, and it waits for approval and for the user's confirmation. It is refused if a later change touched the same flows.

The approval itself is kept honest. A fingerprint of the request is checked again when it is applied, so a request edited after it was asked for is refused. A request expires after an hour, is applied once even if two processes try, and five wrong tokens in a row pause approval for five minutes. Every step, from a request to an approval, a refusal or a failure, is written to an audit log, so the operator can review what an agent asked for and what was done. If the server dies half way through a change, the next start settles it from the flow table.

Which agents it works with

In our setup the agent is the central agent of a multi-domain network orchestrator. It runs a 4-bit Qwen3-30B model, served locally on a single NVIDIA RTX A5000 GPU with 24 GB of memory, and it talks to the controllers only through MCP.

Nothing in the server depends on that model. It speaks the standard Model Context Protocol, over SSE or over standard input and output, so any MCP client can use it, including agents built on hosted models such as Claude or GPT. Connecting one takes the address of the server, or the command that starts it (`run_ryu_mcp.py`), and nothing else. The safeguards live in the server and not in the model, which is what makes it a security layer for any agent and not a feature of one: a hosted model meets the same queue, the same ratings and the same approval as our local one. The user's confirmation needs a client that implements elicitation. A client that does not can still use the server, and then the operator's approval alone stands, or the write is refused if the server runs strict. We tried only the local model in the chat tests, and the tests on the server itself

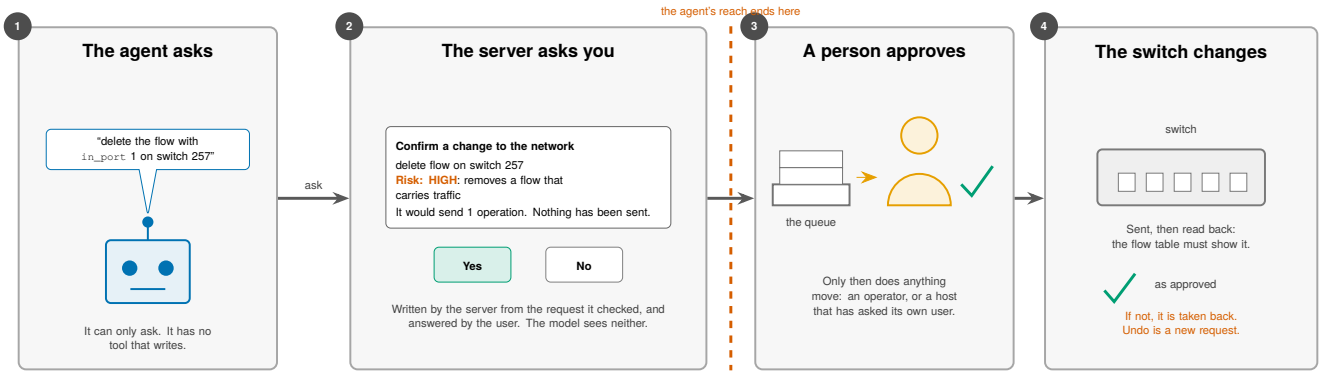

Figure 1: The write path. The agent can only ask (1). The server puts the change to the user in its own words (2), and nothing moves until an operator decides (3). What was sent is then read back from the switch, and undone if it is not what was approved (4).

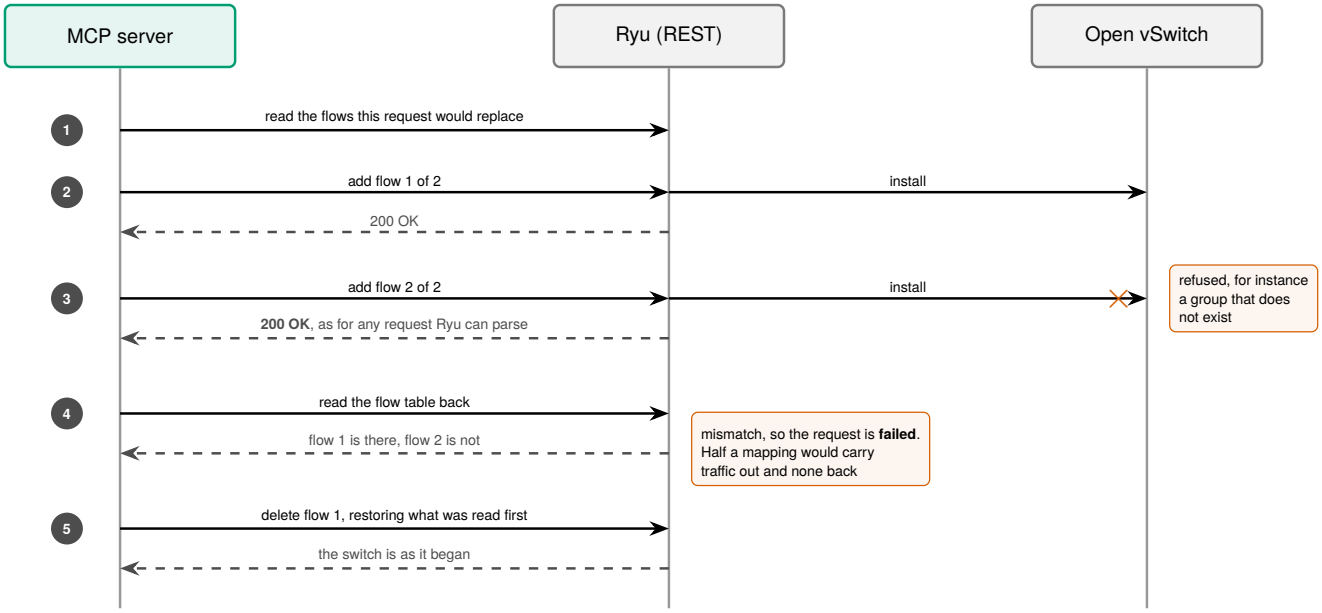

Figure 2: A VLAN mapping of two flows in which the switch refuses the second. Ryu answers 200 to both, so only reading the flow table back shows it, and the flow that landed is taken back.

used scripted and random callers, so hosted models are untested.

How we tested it, and what we found

We tried to break the server in four ways: unit tests against a fake of Ryu's REST API that behaves like a live one, 15 scripted attacks plus a fuzzer of 25,000 random calls, mutation testing that switches each guard off in turn, and a stress run on a live lab Ryu with three Open vSwitch switches, which ended by undoing every change and comparing the flow tables with the start.

No write reached a switch without a decision, in the attacks or the fuzzing, and every approved request was applied. The attacks included arguments that look like consent, a wildcard drop-all, a burst of 300 requests, a queue entry rewritten after it was asked for, and 100 kB token guesses. On the live Ryu, 64 callers made 638 reads with no errors, and a flood of 320 write requests queued exactly the hourly cap of 20. Twenty requests approved at once from two paths were each applied exactly once, and undoing all of them restored the 34 flows exactly.

The tests found more about our own code than the attacks did. The first mutation run missed four of 17 guards, and all 29 are now caught. A live controller showed what the fake had hidden: four requests that Ryu accepted never reached the switch, and an action written in lower case installed a flow with no actions, which drops traffic. A process killed in the middle of a change left it claimed forever, which is why the server now recovers it. These are fixed and covered by tests.

What it does not cover

No language model tried to outwit the standalone server, and the user's confirmation was tested with test clients and not a production agent host. Approval is only as good as the person approving, and whoever can write the server's state directory is in effect the operator. An undo is exact only for what can be read back. Read-back shows the switch holds the flow, not that traffic does what the person wanted. The scope is OpenFlow 1.3 through one version of Ryu.

Conclusion

The server is a security layer between AI agents and the network. It leaves reading open, and puts every change under the scrutiny of an operator and of the user: rated, put to the user in the server's own words, approved, checked on the switch and written to an audit log. An agent that can only ask cannot make an irreversible change on its own. Nothing reached a switch without a decision in any test, and every change made to a live network was taken back exactly.

References

[1] Anthropic, Model Context Protocol (2024).

[2] Ryu SDN Framework, `ryu-sdn.org`.

[3] Open Networking Foundation, OpenFlow Switch Specification, v1.3.

[4] Model Context Protocol, Specification: Elicitation (2025).